

Diseño, Calibración y Validación de un Sistema de Plausibilidad APPS

Héctor Enrique Camacho Duarte
Data Acquisition – Motor Sports
Facultad de Ingeniería, UNAM

Resumen—Se diseñó y validó un sistema redundante de sensores de posición del pedal del acelerador (APPS) basado en dos potenciómetros de 10 k Ω . El diseño emplea rangos eléctricos asimétricos, normalización independiente a una escala común de 0–100 %, verificación de plausibilidad con una tolerancia máxima del 10 % y confirmación temporal de falla de 100 ms.

El desarrollo incluyó el cálculo de las redes resistivas, la elaboración del esquemático en Altium Designer, simulación en Tinkercad utilizando Arduino Uno, desarrollo iterativo del firmware, implementación física en protoboard con ESP32 y adaptación de un dashboard desarrollado en Python/PyQt5 para la visualización y diagnóstico de ambos canales APPS en tiempo real.

El funcionamiento se validó mediante revisión del esquemático, simulación y pruebas físicas utilizando telemetría serial y el dashboard.

I. REQUERIMIENTOS Y METODOLOGÍA

El objetivo del sistema fue representar el comportamiento de dos canales APPS independientes que se desplazan simultáneamente cuando el piloto acciona el pedal del acelerador, pero que entregan rangos de tensión diferentes.

Esta asimetría permite contar con redundancia y facilitar la detección de condiciones eléctricas anómalas.

Para comparar ambos canales, sus lecturas son convertidas de manera independiente a una escala común de 0–100 %. Se adoptó una tolerancia de plausibilidad del 10 %; por lo tanto, cuando la diferencia entre ambos canales supera dicho valor se genera una condición de falla.

Para evitar falsos positivos provocados por ruido o eventos transitorios, la falla debe permanecer activa continuamente durante al menos 100 ms antes de deshabilitar de forma definitiva el motor.

El proceso de desarrollo se realizó mediante las siguientes etapas:

1. Cálculo de las resistencias necesarias para generar dos rangos eléctricos asimétricos.
2. Elaboración del esquemático eléctrico en Altium Designer.
3. Revisión de conectividad y reglas de diseño del esquemático.
4. Implementación inicial del circuito en Tinkercad utilizando un Arduino Uno.
5. Elaboración de un pseudocódigo para establecer la lógica de funcionamiento del sistema.
6. Desarrollo iterativo del firmware mediante un proceso de prueba, medición y corrección.
7. Calibración experimental de los dos canales APPS.
8. Implementación del circuito físico en protoboard.

9. Migración del código del Arduino Uno a los pines y ADC del ESP32.
10. Adaptación de un dashboard previamente utilizado para visualizar la velocidad de caída de un satélite, modificándolo para monitorear los dos canales APPS, sus porcentajes y su diferencia de plausibilidad.

II. DISEÑO ELÉCTRICO Y CÁLCULO DE RESISTENCIAS

Para representar cada sensor APPS se utilizó un potenciómetro lineal de 10 k Ω . Con el objetivo de obtener rangos eléctricos diferentes, se colocaron resistencias fijas en los extremos de cada potenciómetro.

Considerando un potenciómetro de resistencia total R_P , dos resistencias extremas iguales R_S y una alimentación de $V_{CC} = 5$ V, el voltaje mínimo y máximo disponible en el cursor del potenciómetro puede calcularse mediante:

$$V_{\min} = V_{CC} \frac{R_S}{2R_S + R_P} \quad (1)$$

$$V_{\max} = V_{CC} \frac{R_S + R_P}{2R_S + R_P} \quad (2)$$

II-A. Cálculo del Canal 1

Para el canal APPS1 se utilizaron dos resistencias de 1 k Ω y un potenciómetro de 10 k Ω .

El voltaje mínimo teórico es:

$$V_{\min,1} = 5 \frac{1}{1 + 10 + 1} = 0.417 \text{ V} \quad (3)$$

El voltaje máximo teórico es:

$$V_{\max,1} = 5 \frac{1 + 10}{1 + 10 + 1} = 5 \frac{11}{12} = 4.583 \text{ V} \quad (4)$$

II-B. Cálculo del Canal 2

Para el canal APPS2 se utilizaron dos resistencias de 5 k Ω y un potenciómetro de 10 k Ω .

El voltaje mínimo teórico es:

$$V_{\min,2} = 5 \frac{5}{5 + 10 + 5} = 1.250 \text{ V} \quad (5)$$

El voltaje máximo teórico es:

$$V_{\max,2} = 5 \frac{5 + 10}{5 + 10 + 5} = 5 \frac{15}{20} = 3.750 \text{ V} \quad (6)$$

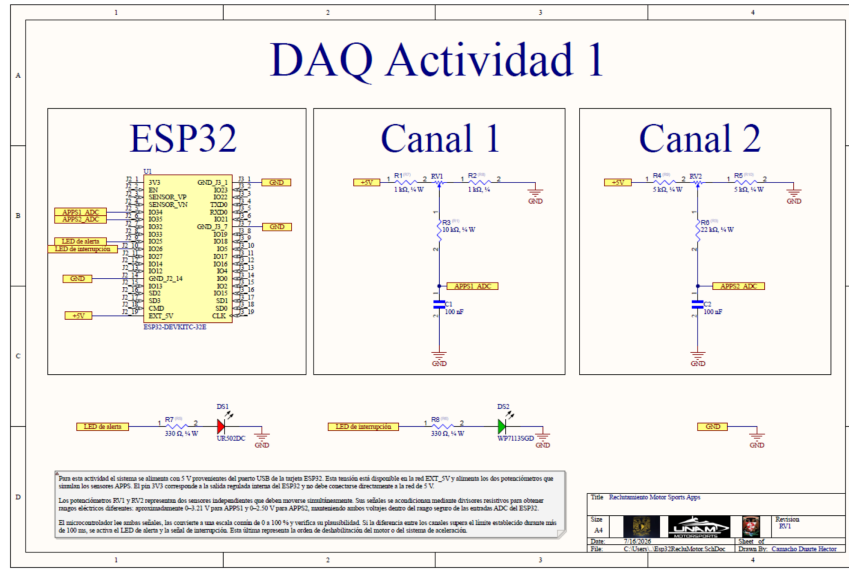


Figura 1. Esquemático completo del sistema de validación de plausibilidad APPS desarrollado en Altium Designer.

De esta manera se obtuvieron dos señales crecientes, pero con rangos eléctricos diferentes. La asimetría permite diferenciar ambos canales y detectar lecturas incompatibles con el comportamiento esperado del sistema.

El diseño completo fue posteriormente trasladado a Altium Designer, donde se realizó el esquemático eléctrico. En esta etapa se definieron las conexiones correspondientes a los dos canales APPS, las redes resistivas, los filtros RC, las entradas del microcontrolador y las señales correspondientes a los indicadores.

El esquemático desarrollado se presenta en la Fig. 1.

III. FILTROS DE ENTRADA

Después de establecer los rangos eléctricos de los potenciómetros, cada señal fue conducida hacia la entrada ADC mediante un filtro RC pasa-bajas de primer orden.

Es importante diferenciar las funciones de las resistencias presentes en el circuito.

Las resistencias de 1 kΩ del Canal 1 y de 5 kΩ del Canal 2 se encuentran en los extremos de los potenciómetros y permiten establecer los diferentes rangos eléctricos de ambos sensores.

Por otra parte, las resistencias de 10 kΩ y 22 kΩ, junto con los capacitores de 100 nF, corresponden a las etapas de filtrado conectadas entre el cursor de cada potenciómetro y la correspondiente entrada ADC.

III-A. Filtro del Canal 1

En el Canal 1 se utilizó una resistencia serie $R_3 = 10\text{ k}\Omega$ y un capacitor $C_1 = 100\text{ nF}$ conectado entre el nodo APPS1_ADC y tierra.

La configuración simplificada se muestra en la Fig. III-A.

Filtro RC utilizado entre el cursor del Canal 1 y la entrada APPS1 ADC.

La frecuencia de corte aproximada de un filtro RC se calcula mediante:

$$f_c = \frac{1}{2\pi RC} \quad (7)$$

Para el Canal 1:

$$f_{c1} = \frac{1}{2\pi(10\text{ k}\Omega)(100\text{ nF})} \quad (8)$$

por lo que:

$$f_{c1} \approx 159.15\text{ Hz} \quad (9)$$

III-B. Filtro del Canal 2

En el Canal 2 se utilizó una resistencia serie $R_6 = 22\text{ k}\Omega$ y un capacitor $C_2 = 100\text{ nF}$ conectado entre el nodo APPS2_ADC y tierra.

La configuración simplificada se muestra en la Fig. III-B.

Filtro RC utilizado entre el cursor del Canal 2 y la entrada APPS2 ADC.

Para el Canal 2:

$$f_{c2} = \frac{1}{2\pi(22\text{ k}\Omega)(100\text{ nF})} \quad (10)$$

por lo que:

$$f_{c2} \approx 72.34\text{ Hz} \quad (11)$$

La función de estas etapas es reducir la influencia de ruido eléctrico y atenuar transitorios rápidos antes de realizar la adquisición mediante el ADC.

Debido a que la posición mecánica del pedal presenta variaciones considerablemente más lentas, el filtrado permite

Cuadro I
CALIBRACIÓN FINAL DE LOS CANALES APPS

Canal	0 %	100 %
APPS1	0.415 V	4.585 V
APPS2	1.251 V	3.749 V

mejorar la estabilidad de la lectura sin afectar de forma significativa el seguimiento del movimiento del acelerador.

Las frecuencias calculadas corresponden a una aproximación basada en la resistencia serie de cada filtro y el capacitor asociado.

IV. PROCESO DE CALIBRACIÓN

Una vez construido el circuito en Tinkercad, se verificaron los voltajes utilizando multímetros virtuales y las lecturas obtenidas por el ADC del microcontrolador.

Durante las primeras pruebas se observaron diferencias entre las lecturas del multímetro y las obtenidas mediante el Arduino.

El problema se localizó en la red conectada a la entrada del ADC, donde una resistencia adicional estaba generando un efecto de carga y modificando los voltajes esperados.

Se corrigió la configuración de cada canal manteniendo la red resistiva diseñada para establecer el rango del potenciómetro y la etapa RC correspondiente para el filtrado.

Posteriormente se realizaron nuevamente las mediciones directamente en los nodos asociados a las entradas ADC.

Los valores finales de calibración se muestran en la Tabla I.

Los resultados experimentales fueron muy cercanos a los valores obtenidos mediante el cálculo teórico.

Para APPS1 se calculó un rango teórico de 0.417 V–4.583 V, mientras que experimentalmente se obtuvieron 0.415 V–4.585 V.

Para APPS2 se calculó un rango teórico de 1.250 V–3.750 V, obteniéndose experimentalmente 1.251 V–3.749 V.

Los valores experimentales fueron utilizados posteriormente como los límites de calibración de cada sensor dentro del firmware.

V. NORMALIZACIÓN Y PLAUSIBILIDAD

Debido a que ambos sensores utilizan rangos eléctricos diferentes, la comparación no se realiza directamente entre sus voltajes.

Cada señal es transformada primero a una escala común de 0–100 %.

La normalización se realiza mediante:

$$p_i = 100 \frac{V_i - V_{\min,i}}{V_{\max,i} - V_{\min,i}} \quad (12)$$

donde p_i representa la posición porcentual calculada para cada sensor APPS.

Los resultados se limitan al intervalo:

$$0 \leq p_i \leq 100 \quad (13)$$

Posteriormente se calcula la diferencia absoluta entre los dos canales:

$$\Delta p = |p_1 - p_2| \quad (14)$$

El criterio de plausibilidad utilizado fue:

$$\Delta p \leq 10 \% \Rightarrow \text{Sistema OK} \quad (15)$$

mientras que:

$$\Delta p > 10 \% \Rightarrow \text{Falla de plausibilidad} \quad (16)$$

Cuando la diferencia supera el 10 %, el sistema inicia un temporizador.

Si la condición desaparece antes de 100 ms, la falla se considera transitoria y el sistema regresa al estado de operación normal.

Si la condición permanece continuamente durante al menos 100 ms, la falla se confirma y queda enclavada.

Como acción de seguridad, el sistema deshabilita la señal de habilitación del motor y enciende el indicador correspondiente a una condición de falla.

VI. PSEUDOCÓDIGO DEL SISTEMA

Antes del desarrollo del firmware se elaboró un pseudocódigo para establecer la secuencia de adquisición, procesamiento de datos y respuesta ante condiciones de falla.

```

1  INICIALIZAR:
2  ADC
3  APPS1
4  APPS2
5  LED verde
6  LED rojo
7  Motor
8  Interrupcion
9  Comunicacion Serial
10
11 MIENTRAS sistema activo:
12
13 Leer APPS1
14 Leer APPS2
15
16 Convertir ADC a voltaje
17
18 Normalizar APPS1 a 0-100 %
19 Normalizar APPS2 a 0-100 %
20
21
22 Calcular:
23 DIF = ABS (APPS1 % - APPS2 %)
24
25 Revisar APPS1:
26 Corto a GND
27 Corto a VCC
28 Fuera de rango
29
30 Revisar APPS2:
31 Corto a GND
32 Corto a VCC
33 Fuera de rango
34
35 SI DIF > 10 %:
36 Declarar falla instantanea
37
38 SI existe falla:
39 Iniciar temporizador
40
41 SI falla dura >= 100 ms:
42 Confirmar falla
43 Enclavar falla
44
45 Motor = OFF
46 LED rojo = ON

```

```

47     LED verde = OFF
48
49     SI existe cortocircuito:
50         Interruccion = HIGH
51
52     SI sistema OK:
53         Motor = ON
54         LED verde = ON
55         LED rojo = OFF
56
57     Enviar datos por Serial
58
59     Revisar comando RESET
60
61 FIN

```

Listing 1. Pseudocódigo general del sistema APPS

VII. CÓDIGO DE SENSORES

La primera versión del programa fue implementada para Arduino Uno durante las pruebas realizadas en Tinkercad.

El desarrollo se realizó de forma iterativa mediante un proceso de prueba y error, comparando las lecturas obtenidas por el ADC con los valores observados mediante multímetros.

Durante este proceso se modificaron los rangos de calibración, los límites eléctricos y la red de acondicionamiento hasta conseguir que las mediciones obtenidas mediante el microcontrolador coincidieran con las observadas directamente en el circuito.

Una vez validado el funcionamiento lógico, el circuito se implementó físicamente en protoboard y el firmware fue migrado a un ESP32-WROOM-32D.

En la versión final se utilizaron GPIO34 y GPIO35 para la adquisición de APPS1 y APPS2, respectivamente.

GPIO23 fue utilizado para la señal de interrupción, GPIO22 para la habilitación del motor y GPIO25 y GPIO26 para los indicadores LED.

Los parámetros principales utilizados por el firmware se pueden representar mediante el siguiente fragmento:

```

1 // Pines de adquisicion
2 #define PIN_APPS1 34
3 #define PIN_APPS2 35
4
5 // Salidas digitales
6 #define PIN_INTERRUPCION 23
7 #define PIN_MOTOR_ENABLE 22
8
9
10 #define PIN_LED_ROJO 25
11 #define PIN_LED_VERDE 26
12
13 // Calibracion APPS1
14 const float APPS1_MIN_V = 0.415;
15 const float APPS1_MAX_V = 4.585;
16
17 // Calibracion APPS2
18 const float APPS2_MIN_V = 1.251;
19 const float APPS2_MAX_V = 3.749;
20
21 // Tolerancia maxima
22 const float TOLERANCIA_PLAUSIBILIDAD =
23     10.0;
24
25 // Tiempo de confirmacion
26 const unsigned long
27 TIEMPO_CONFIRMACION_FALLA = 100;

```

Listing 2. Parámetros principales del sistema APPS

La normalización y comparación de ambos canales se realiza de forma continua.

```

1 // Normalizar ambos sensores
2 apps1Pct =
3     normalizarAPPS1(apps1Voltaje);
4
5 apps2Pct =
6     normalizarAPPS2(apps2Voltaje);
7
8 // Calcular diferencia absoluta
9 diff =
10     abs(apps1Pct - apps2Pct);
11
12 // Verificar diferencia
13 if (diff > 10.0) {
14     fallaInstantanea = true;
15
16 }
17
18 // Confirmar falla si persiste
19 // durante al menos 100 ms
20 if (
21     fallaInstantanea &&
22     millis() - tiempoInicioFalla >= 100
23 ) {
24     fallaEnclavada = true;
25 }
26
27 }
28
29 }

```

Listing 3. Lógica simplificada de normalización y plausibilidad

El firmware contempla tres estados principales:

- Sistema OK.
- Falla pendiente.
- Falla confirmada y enclavada.

También permite detectar cortocircuitos hacia GND o VCC, valores fuera del rango eléctrico esperado y fallas de plausibilidad entre ambos sensores.

Cuando se detecta un cortocircuito, se activa la salida digital correspondiente a la señal de interrupción.

VIII. SIMULACIÓN Y PROTOTIPO FÍSICO

Después de realizar el esquemático en Altium Designer, se construyó una primera implementación funcional en Tinkercad utilizando un Arduino Uno.

La simulación permitió verificar progresivamente el comportamiento de los potenciómetros, las redes resistivas, los filtros de entrada, las lecturas ADC y la lógica implementada en el firmware.

La implementación utilizada durante esta etapa se presenta en la Fig. 2.

La simulación permitió verificar los rangos de tensión de ambos sensores, realizar el proceso de calibración y validar la lógica de plausibilidad antes de construir el prototipo físico.

Se probaron diferentes posiciones del recorrido, incluyendo valores cercanos al 0 %, 50 % y 100 %, así como condiciones deliberadas donde la diferencia entre APPS1 y APPS2 superaba el 10 %.

En operación normal, ambos canales aumentan simultáneamente en porcentaje, aunque sus voltajes sean diferentes debido a la asimetría eléctrica implementada.

La Tabla II presenta algunos puntos de referencia obtenidos después de la calibración.

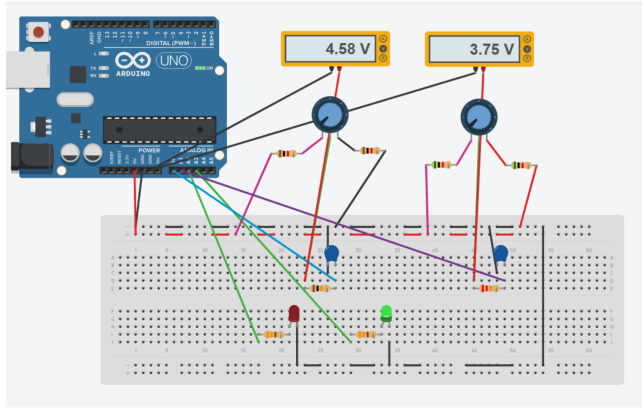


Figura 2. Simulación del sistema APPS implementada en Tinkercad utilizando Arduino Uno, dos potenciómetros y monitoreo de las señales.

Cuadro II
VALORES EQUIVALENTES DE LOS DOS CANALES APPS

Posición	APPS1	APPS2
0 %	0.415 V	1.251 V
50 %	2.500 V	2.500 V
100 %	4.585 V	3.749 V

Después de obtener un comportamiento funcional en simulación, el circuito fue construido en una protoboard y el programa fue migrado al ESP32.

La validación se realizó en tres etapas:

1. Verificación del esquemático y las reglas de diseño en Altium Designer.
2. Simulación funcional del circuito en Tinkercad.
3. Validación física mediante ESP32, protoboard, comunicación serial y dashboard de monitoreo.

IX. CÓDIGO PYTHON Y DASHBOARD

Como etapa final se adaptó un dashboard desarrollado previamente para visualizar datos relacionados con la velocidad de caída de un satélite.

La arquitectura de dicha interfaz se modificó para crear un sistema de visualización específico para el monitoreo APPS.

El dashboard fue desarrollado en Python utilizando PyQt5 y recibe la telemetría enviada por el ESP32 mediante comunicación serial a 115 200 baud.

La interfaz final utilizada durante las pruebas se presenta en la Fig. 3.

Entre las principales funcionalidades del dashboard se encuentran:

- Visualización independiente de APPS1 y APPS2.
- Indicadores circulares de posición de 0–100 %.
- Visualización del voltaje de cada sensor.
- Visualización de las lecturas adquiridas.
- Gráfica temporal simultánea de ambos canales.
- Visualización de la diferencia porcentual.
- Barra gráfica de plausibilidad.



Figura 3. Dashboard de monitoreo del sistema APPS desarrollado en Python utilizando PyQt5.

- Indicador del estado general del sistema.
- Indicador de falla de plausibilidad.
- Estado de habilitación o deshabilitación del motor.
- Estado de la señal de interrupción.
- Visualización del tipo de falla detectada.
- Monitor de comunicación serial.
- Selección del puerto de comunicación.
- Configuración del baudrate.
- Envío de comandos hacia el microcontrolador.
- Cambio entre tema oscuro y tema claro.

El dashboard también incluye un modo de simulación que permite realizar pruebas específicas para:

- Operación normal.
- Falla temporal.
- Falla de plausibilidad.
- Cortocircuito en APPS1.
- Cortocircuito en APPS2.
- Reinicio del sistema.

El gestor serial recibe continuamente las líneas transmitidas por el ESP32 y posteriormente interpreta los valores de APPS1, APPS2, diferencia porcentual, estado, interrupción, motor y tipo de falla.

Un ejemplo simplificado del procesamiento realizado en Python es:

```

1
2 # Leer telemetría recibida
3 # mediante el puerto serial
4
5 line = (
6     serial_port
7     .readline()
8     .decode()
9     .strip()
10 )
11
12 # Interpretar la trama
13 # enviada por el ESP32
14
15 data = parse_apps_line(line)
16
17 # Actualizar indicador APPS1
18
19 gauge1.set_value (

```

```

20 data['apps1_pct'],
21 data['apps1_raw'],
22 data['apps1_v']
23 )
24
25 # Actualizar indicador APPS2
26
27 gauge2.set_value(
28     data['apps2_pct'],
29     data['apps2_raw'],
30     data['apps2_v']
31 )
32
33 # Mostrar diferencia
34 # entre ambos canales
35
36 plaus_bar.set_difference(
37     data['diff']
38 )
39
40 # Actualizar estado general
41
42 status_panel.update_status(
43     data['state'],
44     data['interrupt'],
45     data['motor'],
46     data['fault']
47 )

```

Listing 4. Procesamiento de telemetría APPS en el dashboard

La interfaz permite observar en tiempo real cómo ambos sensores incrementan su porcentaje conforme cambia la posición del pedal y permite visualizar directamente cualquier diferencia entre ambos.

En condiciones normales, los indicadores de APPS1 y APPS2 presentan valores porcentuales similares.

Cuando la diferencia supera el 10 %, la interfaz permite identificar visualmente la discrepancia y el estado reportado por el sistema de seguridad.

X. VALIDACIÓN DEL SISTEMA

El funcionamiento del sistema fue validado mediante tres métodos complementarios.

En primer lugar, el esquemático realizado en Altium Designer fue revisado mediante las herramientas de comprobación de reglas y conectividad.

Esta etapa permitió verificar la correcta interconexión entre los elementos antes de avanzar a la implementación funcional.

En segundo lugar, el circuito fue implementado en Tinkercad, donde se validaron los rangos eléctricos, la normalización, la tolerancia del 10 %, la persistencia de falla de 100 ms, las salidas de seguridad y el comportamiento de los indicadores.

Las pruebas de funcionamiento normal se realizaron en diferentes puntos del recorrido de los sensores.

También se provocaron diferencias superiores al 10 % para verificar la detección de una falla de plausibilidad.

Cuando la diferencia permanecía activa durante al menos 100 ms, el sistema cambiaba al estado de falla confirmada, deshabilitando la señal correspondiente al motor y activando el indicador visual de falla.

También se validaron condiciones eléctricas anómalas mediante la simulación de cortocircuitos hacia GND y VCC, verificando la activación de la señal de interrupción correspondiente.

Finalmente, el circuito fue implementado físicamente en protoboard utilizando el ESP32.

La adquisición de los sensores fue monitoreada mediante el dashboard desarrollado en Python, permitiendo observar en tiempo real los porcentajes de APPS1 y APPS2, su diferencia y el estado general del sistema.

De esta manera, el diseño fue validado a nivel de esquemático, simulación y prototipo físico con visualización de telemetría.

XI. CONCLUSIONES

Se desarrolló y validó un sistema de plausibilidad APPS basado en dos sensores redundantes con rangos eléctricos asimétricos, normalizados a una escala común de 0–100 %. Los valores experimentales obtenidos mostraron una buena correspondencia con los valores teóricos de diseño. Asimismo, se implementaron filtros RC, una tolerancia máxima del 10 % con 100 ms de persistencia y mecanismos de detección de fallas eléctricas. La validación mediante simulación, implementación física con ESP32 y visualización en un dashboard permitió comprobar el correcto funcionamiento general del sistema APPS.